```
import { useState, useEffect, useRef, useCallback } from "react";

// ─── Palette & Design Tokens ──────────────────────────────────────────
const T = {
  bgDeep:     "#1a0933",
  bgMid:      "#2d1254",
  bgSoft:     "#3d1a6e",
  lavender:   "#9b72cf",
  lavLight:   "#c4a8f0",
  blush:      "#f0a0c8",
  blushDeep:  "#d9609a",
  pink:       "#f472b6",
  pinkLight:  "#fce7f3",
  white:      "#fdf4ff",
  muted:      "#a78bba",
  mutedDark:  "#6b4f8a",
  green:      "#6ee7b7",
  red:        "#fca5a5",
  yellow:     "#fde68a",
  cableOk:    "#c084fc",
  cableBad:   "#475569",
  glow:       "rgba(196,168,240,0.35)",
  glowPink:   "rgba(244,114,182,0.4)",
};

const GRAD = {
  btn:    "linear-gradient(135deg, #9b72cf 0%, #d9609a 100%)",
  btnHov: "linear-gradient(135deg, #b48ee8 0%, #e878b8 100%)",
  bg:     "linear-gradient(160deg, #1a0933 0%, #2d1254 50%, #3a1060 100%)",
  header: "linear-gradient(90deg, rgba(155,114,207,0.18) 0%, rgba(217,96,154,0.12
  card:   "linear-gradient(145deg, rgba(61,26,110,0.95) 0%, rgba(45,18,84,0.98) 1
  cable:  "linear-gradient(90deg, #9b72cf, #f472b6)",
};

// ─── Device Config ────────────────────────────────────────────────────
const DTYPE = { COMPUTER:"computer", SERVER:"server", PRINTER:"printer", SWITCH:"

const DCONFIG = {
  computer:  { icon:"🖥️",  label:"PC",       color:"#7c3aed", border:"#c4a8f0", s
  server:    { icon:"🗄️",  label:"Server",   color:"#5b21b6", border:"#a78bfa", s
  printer:   { icon:"🖨️",  label:"Printer",  color:"#6d28d9", border:"#c4b5fd", s
  switch:    { icon:"🔀",  label:"Switch",   color:"#1e3a5f", border:"#60a5fa", s
  router:    { icon:"📡",  label:"Router",   color:"#3b0764", border:"#e879f9", s
  firewall:  { icon:"🛡️",  label:"FW",       color:"#7f1d1d", border:"#fca5a5", s
```

```
  internet:  { icon:"🌐",  label:"Internet", color:"#064e3b", border:"#6ee7b7", s
  vpn:       { icon:"🔒",  label:"VPN",      color:"#78350f", border:"#fde68a", s
};

function connKey(a,b){ return [a,b].sort().join("~~"); }

// ── Levels ──────────────────────────────────────────────────────────────
const LEVELS = [
  {
    id:1, title:"Office LAN", badge:"🏢",
    subtitle:"Build your first local network",
    story:"The Aoun Tech office just moved in. No cables, no network — just boxes
    objective:"Connect all 4 PCs through the switch",
    timeLimit:90, xp:120,
    devices:[
      {id:"sw1", type:DTYPE.SWITCH,   x:340, y:220, label:"Core-SW",  ip:null,
      {id:"pc1", type:DTYPE.COMPUTER, x:120, y:90,  label:"PC-Alice", ip:"192.168
      {id:"pc2", type:DTYPE.COMPUTER, x:560, y:90,  label:"PC-Bob",   ip:"192.168
      {id:"pc3", type:DTYPE.COMPUTER, x:120, y:350, label:"PC-Carol", ip:"192.168
      {id:"pc4", type:DTYPE.COMPUTER, x:560, y:350, label:"PC-Dave",  ip:"192.168
      {id:"prn", type:DTYPE.PRINTER,  x:340, y:380, label:"Printer",  ip:"192.168
    ],
    required:[["pc1","sw1"],["pc2","sw1"],["pc3","sw1"],["pc4","sw1"],["prn","sw1
    hint:"Select a device, then click another to run a cable between them.",
  },
  {
    id:2, title:"Dual-Network Routing", badge:"🌉",
    subtitle:"Bridge two departments",
    story:"Engineering and Marketing refuse to share a switch. Your job: keep bot
    objective:"Connect both switches to the router, devices to their switch",
    timeLimit:120, xp:200,
    devices:[
      {id:"rt1",  type:DTYPE.ROUTER,   x:340, y:190, label:"Core-RT",   ip:"10.0.
      {id:"sw1",  type:DTYPE.SWITCH,   x:150, y:330, label:"Eng-SW",    ip:null,
      {id:"sw2",  type:DTYPE.SWITCH,   x:530, y:330, label:"Mkt-SW",    ip:null,
      {id:"pc1",  type:DTYPE.COMPUTER, x:50,  y:100, label:"Eng-1",     ip:"192.1
      {id:"pc2",  type:DTYPE.COMPUTER, x:200, y:100, label:"Eng-2",     ip:"192.1
      {id:"srv1", type:DTYPE.SERVER,   x:290, y:430, label:"Eng-SRV",   ip:"192.1
      {id:"pc3",  type:DTYPE.COMPUTER, x:430, y:100, label:"Mkt-1",     ip:"192.1
      {id:"pc4",  type:DTYPE.COMPUTER, x:600, y:100, label:"Mkt-2",     ip:"192.1
      {id:"prn",  type:DTYPE.PRINTER,  x:490, y:430, label:"Mkt-PRT",   ip:"192.1
    ],
    required:[["pc1","sw1"],["pc2","sw1"],["srv1","sw1"],["sw1","rt1"],["pc3","sw
    hint:"Each department gets its own switch. The router connects them together.
  },
  {
    id:3, title:"Secure Gateway", badge:"🛡",
```

```
  subtitle:"Deploy firewall & VPN",
  story:"The CEO just read a cybersecurity report. Every packet to the internet
  objective:"Build: Devices→Switches→Router→Firewall→Internet, plus VPN",
  timeLimit:150, xp:320,
  devices:[
    {id:"inet", type:DTYPE.INTERNET,  x:340, y:28,  label:"Internet",  ip:null,
    {id:"fw1",  type:DTYPE.FIREWALL,  x:340, y:120, label:"FW-Edge",   ip:"203.
    {id:"vpn",  type:DTYPE.VPN,       x:200, y:120, label:"VPN-GW",    ip:"203.
    {id:"rt1",  type:DTYPE.ROUTER,    x:340, y:220, label:"Core-RT",   ip:"10.0
    {id:"sw1",  type:DTYPE.SWITCH,    x:190, y:320, label:"Floor1-SW", ip:null,
    {id:"sw2",  type:DTYPE.SWITCH,    x:490, y:320, label:"Floor2-SW", ip:null,
    {id:"pc1",  type:DTYPE.COMPUTER,  x:80,  y:410, label:"PC-F1A",    ip:"10.1
    {id:"pc2",  type:DTYPE.COMPUTER,  x:190, y:410, label:"PC-F1B",    ip:"10.1
    {id:"srv1", type:DTYPE.SERVER,    x:300, y:420, label:"DB-SRV",    ip:"10.1
    {id:"pc3",  type:DTYPE.COMPUTER,  x:420, y:410, label:"PC-F2A",    ip:"10.2
    {id:"srv2", type:DTYPE.SERVER,    x:530, y:410, label:"Web-SRV",   ip:"10.2
    {id:"pc4",  type:DTYPE.COMPUTER,  x:630, y:410, label:"PC-F2B",    ip:"10.2
  ],
  required:[
    ["inet","fw1"],["fw1","vpn"],["fw1","rt1"],
    ["rt1","sw1"],["rt1","sw2"],
    ["pc1","sw1"],["pc2","sw1"],["srv1","sw1"],
    ["pc3","sw2"],["srv2","sw2"],["pc4","sw2"],
  ],
  hint:"Firewall first — then router, then switches. Don't forget the VPN!",
},
{
  id:4, title:"Load-Balanced WAN", badge:"⚖️",
  subtitle:"Scale to enterprise",
  story:"Aoun Corp is going global. You need dual routers for redundancy, a DMZ
  objective:"Full WAN with dual routers, DMZ, and load balancer",
  timeLimit:180, xp:500,
  devices:[
    {id:"inet",  type:DTYPE.INTERNET,  x:340, y:20,  label:"Internet",  ip:null
    {id:"fw1",   type:DTYPE.FIREWALL,  x:340, y:100, label:"Edge-FW",   ip:"203
    {id:"rt1",   type:DTYPE.ROUTER,    x:200, y:200, label:"Router-A",  ip:"10.
    {id:"rt2",   type:DTYPE.ROUTER,    x:480, y:200, label:"Router-B",  ip:"10.
    {id:"sw1",   type:DTYPE.SWITCH,    x:120, y:310, label:"SW-Core1",  ip:null
    {id:"sw2",   type:DTYPE.SWITCH,    x:340, y:310, label:"SW-DMZ",    ip:null
    {id:"sw3",   type:DTYPE.SWITCH,    x:560, y:310, label:"SW-Core2",  ip:null
    {id:"pc1",   type:DTYPE.COMPUTER,  x:40,  y:400, label:"User-1",    ip:"10.
    {id:"pc2",   type:DTYPE.COMPUTER,  x:130, y:400, label:"User-2",    ip:"10.
    {id:"srv1",  type:DTYPE.SERVER,    x:280, y:400, label:"DMZ-Web",   ip:"172
    {id:"srv2",  type:DTYPE.SERVER,    x:380, y:400, label:"DMZ-Mail",  ip:"172
    {id:"pc3",   type:DTYPE.COMPUTER,  x:500, y:400, label:"User-3",    ip:"10.
    {id:"pc4",   type:DTYPE.COMPUTER,  x:600, y:400, label:"User-4",    ip:"10.
  ],
```

```
    required:[
      ["inet","fw1"],
      ["fw1","rt1"],["fw1","rt2"],
      ["rt1","sw1"],["rt1","sw2"],
      ["rt2","sw2"],["rt2","sw3"],
      ["pc1","sw1"],["pc2","sw1"],
      ["srv1","sw2"],["srv2","sw2"],
      ["pc3","sw3"],["pc4","sw3"],
    ],
    hint:"Both routers share the DMZ switch — that's your redundancy path.",
  },
];

// ─── Particle bg ──────────────────────────────────────────────────────────
function Particles() {
  return (
    <div style={{position:"fixed",inset:0,pointerEvents:"none",overflow:"hidden",
      {Array.from({length:18}).map((_,i)=>(
        <div key={i} style={{
          position:"absolute",
          left:`${(i*57+13)%100}%`,
          top:`${(i*43+7)%100}%`,
          width: i%3===0?3:2,
          height:i%3===0?3:2,
          borderRadius:"50%",
          background: i%2===0? T.lavLight : T.blush,
          opacity:0.25+0.2*(i%3),
          animation:`drift${i%4} ${8+i*1.3}s ease-in-out infinite`,
          animationDelay:`${i*0.7}s`,
        }}/>
      ))}
    </div>
  );
}

// ─── Ping Modal ───────────────────────────────────────────────────────────
function PingModal({devices,connections,onClose}){
  const [log,setLog]=useState([]);
  const [busy,setBusy]=useState(false);

  const run=()=>{
    setLog([]); setBusy(true);
    const ends=devices.filter(d=>d.type===DTYPE.COMPUTER||d.type===DTYPE.SERVER||
    const msgs=[];
    for(let i=0;i<ends.length;i++)
      for(let j=i+1;j<ends.length;j++){
        const ok=connections.has(connKey(ends[i].id,ends[j].id));
```

```
        msgs.push({from:ends[i].label,to:ends[j].label,ok,ms:Math.floor(Math.rand
      }
    let idx=0;
    const t=setInterval(()=>{
      if(idx>=msgs.length){clearInterval(t);setBusy(false);return;}
      setLog(l=>[...l,msgs[idx++]]);
    },220);
  };

  return(
    <div style={{position:"fixed",inset:0,background:"rgba(10,0,25,0.75)",display
      <div style={{background:GRAD.card,border:`1px solid ${T.lavender}`,borderRa
        <div style={{display:"flex",justifyContent:"space-between",alignItems:"ce
          <span style={{fontFamily:"'Courier New',monospace",color:T.blush,fontSi
          <button onClick={onClose} style={{background:"none",border:`1px solid $
        </div>
        <div style={{background:"#0d0020",borderRadius:10,padding:"14px 16px",fon
          {log.length===0&&<span style={{color:T.mutedDark}}>$ awaiting command..
          {log.map((m,i)=>(
            <div key={i} style={{color:m.ok?T.green:T.red}}>
              {m.ok?"✓":"✗"}  {m.from} → {m.to}
              {m.ok?<span style={{color:T.muted}}>  {m.ms}ms reply</spa
            </div>
          ))}
        </div>
        <button onClick={run} disabled={busy} style={{width:"100%",padding:"11px
          {busy?"▶▶ PINGING...":"▶  PING ALL DEVICES"}
        </button>
      </div>
    </div>
  );
}

// ─── Traceroute Modal ──────────────────────────────────────────────
function TraceModal({devices,onClose}){
  const [target,setTarget]=useState("");
  const [log,setLog]=useState([]);
  const [busy,setBusy]=useState(false);

  const run=()=>{
    const dev=devices.find(d=>d.label.toLowerCase().includes(target.toLowerCase()
    if(!dev){setLog([{txt:"Host not found.",ok:false}]);return;}
    setBusy(true); setLog([]);
    const hops=[
      {ip:"192.168.0.1",ms:1},{ip:"10.0.0.1",ms:3},{ip:"203.0.113.1",ms:8},
      {ip:dev.ip,ms:12+Math.floor(Math.random()*10)},
    ];
```

```
    let i=0;
    const t=setInterval(()=>{
      if(i>=hops.length){clearInterval(t);setBusy(false);return;}
      setLog(l=>[...l,{hop:i+1,...hops[i],ok:true}]);
      i++;
    },350);
  };

  return(
    <div style={{position:"fixed",inset:0,background:"rgba(10,0,25,0.75)",display
      <div style={{background:GRAD.card,border:`1px solid ${T.blush}`,borderRadiu
        <div style={{display:"flex",justifyContent:"space-between",alignItems:"ce
          <span style={{fontFamily:"'Courier New',monospace",color:T.lavLight,fon
          <button onClick={onClose} style={{background:"none",border:`1px solid $
        </div>
        <div style={{display:"flex",gap:8,marginBottom:12}}>
          <input value={target} onChange={e=>setTarget(e.target.value)} placehold
          <button onClick={run} disabled={busy} style={{padding:"8px 16px",backgr
        </div>
        <div style={{background:"#0d0020",borderRadius:10,padding:"14px 16px",fon
          {log.length===0&&<span style={{color:T.mutedDark}}>$ traceroute &lt;tar
          {log.map((m,i)=>(
            <div key={i} style={{color:m.ok?T.lavLight:T.red}}>
              {m.hop}  {m.ip}  <span style={{color:T.muted}}>
            </div>
          ))}
        </div>
      </div>
    </div>
  );
}

// ─── Level Complete ──────────────────────────────────────────────────────────
function LevelComplete({level,timeLeft,timeLimit,xp,isLast,onNext,onMenu}){
  const pct=Math.round((timeLeft/timeLimit)*100);
  const stars=pct>66?3:pct>33?2:1;
  return(
    <div style={{position:"fixed",inset:0,background:"rgba(10,0,25,0.88)",display
      <div style={{background:GRAD.card,border:`1px solid ${T.lavender}`,borderRa
        <div style={{fontSize:60,marginBottom:10}}>{isLast?"🏆":level.badge}</div
        <h2 style={{fontFamily:"Georgia,serif",color:T.blush,margin:"0 0 6px",fon
          {isLast?"Network Engineering Expert!":"Level Complete!"}
        </h2>
        <p style={{color:T.muted,fontSize:13,margin:"0 0 24px",lineHeight:1.6}}>
          {isLast
            ? "Outstanding! You've mastered switching, routing, firewalls, VPNs a
            : `Level ${level.id} — ${level.title} — cleared!`}
```

```
        </p>
        <div style={{fontSize:34,marginBottom:16,letterSpacing:6}}>{"⭐".repeat(s
        <div style={{background:"rgba(0,0,0,0.35)",borderRadius:12,padding:"14px
          <div><div style={{color:T.muted,fontSize:10,marginBottom:3}}>TIME LEFT<
          <div><div style={{color:T.muted,fontSize:10,marginBottom:3}}>SCORE</div
          <div><div style={{color:T.muted,fontSize:10,marginBottom:3}}>XP</div>+{
        </div>
        <div style={{display:"flex",gap:12,justifyContent:"center"}}>
          <button onClick={onMenu} style={{padding:"11px 26px",border:`1px solid
          {!isLast&&<button onClick={onNext} style={{padding:"11px 26px",backgrou
        </div>
      </div>
    </div>
  );
}

// ── Story Card ──────────────────────────────────────────────────────────
function StoryCard({level,onStart}){
  return(
    <div style={{position:"fixed",inset:0,background:"rgba(10,0,25,0.88)",display
      <div style={{background:GRAD.card,border:`1px solid ${T.lavender}`,borderRa
        <div style={{fontSize:52,marginBottom:12,textAlign:"center"}}>{level.badg
        <h2 style={{fontFamily:"Georgia,serif",color:T.blush,margin:"0 0 6px",tex
        <p style={{color:T.lavLight,fontSize:12,textAlign:"center",margin:"0 0 20
        <div style={{background:"rgba(0,0,0,0.3)",borderRadius:12,padding:"16px 2
          <p style={{color:T.muted,fontSize:13,lineHeight:1.7,margin:0}}>{level.s
        </div>
        <div style={{background:"rgba(155,114,207,0.1)",borderRadius:10,padding:"
          <span style={{color:T.blush,fontWeight:700}}>🎯 GOAL:</span> {level.obj
        </div>
        <button onClick={onStart} style={{width:"100%",padding:"13px 0",backgroun
          START MISSION
        </button>
      </div>
    </div>
  );
}

// ── Main ────────────────────────────────────────────────────────────────
export default function AounNetworkConnection(){
  const [screen,setScreen]=useState("menu");
  const [levelIdx,setLevelIdx]=useState(0);
  const [devices,setDevices]=useState([]);
  const [connections,setConnections]=useState(new Set());
  const [selected,setSelected]=useState(null);
  const [dragging,setDragging]=useState(null);
  const [dragOff,setDragOff]=useState({x:0,y:0});
```

```
const [timeLeft,setTimeLeft]=useState(0);
const [showStory,setShowStory]=useState(false);
const [showPing,setShowPing]=useState(false);
const [showTrace,setShowTrace]=useState(false);
const [showComplete,setShowComplete]=useState(false);
const [hintDismissed,setHintDismissed]=useState(false);
const [totalXP,setTotalXP]=useState(0);
const svgRef=useRef(null);
const timerRef=useRef(null);

const level=LEVELS[levelIdx];

const initLevel=useCallback((idx)=>{
  const lvl=LEVELS[idx];
  setDevices(lvl.devices.map(d=>({...d})));
  setConnections(new Set());
  setSelected(null); setDragging(null);
  setTimeLeft(lvl.timeLimit);
  setShowComplete(false); setShowPing(false); setShowTrace(false);
  setHintDismissed(false);
  setLevelIdx(idx);
  setScreen("game");
},[]);

// timer
useEffect(()=>{
  if(screen!=="game"||showComplete||showStory)return;
  timerRef.current=setInterval(()=>{
    setTimeLeft(t=>{if(t<=1){clearInterval(timerRef.current);return 0;}return t
  },1000);
  return()=>clearInterval(timerRef.current);
},[screen,showComplete,showStory]);

// win check
useEffect(()=>{
  if(screen!=="game")return;
  const done=level.required.every(([a,b])=>connections.has(connKey(a,b)));
  if(done&&level.required.length>0){
    clearInterval(timerRef.current);
    const earned=Math.round(level.xp*(timeLeft/level.timeLimit));
    setTotalXP(x=>x+earned);
    setTimeout(()=>setShowComplete(true),500);
  }
},[connections,screen,level,timeLeft]);

// svg coords
const svgPos=e=>{
```

```
  const r=svgRef.current?.getBoundingClientRect();
  if(!r)return{x:0,y:0};
  return{x:e.clientX-r.left,y:e.clientY-r.top};
};

const onDevMD=(e,id)=>{
  e.stopPropagation();
  const pos=svgPos(e);
  const dev=devices.find(d=>d.id===id);
  setDragging(id);
  setDragOff({x:pos.x-dev.x,y:pos.y-dev.y});
};
const onSvgMM=e=>{
  if(!dragging)return;
  const p=svgPos(e);
  setDevices(ds=>ds.map(d=>d.id===dragging?{...d,x:p.x-dragOff.x,y:p.y-dragOff.
};
const onSvgMU=()=>setDragging(null);
const onDevClick=(e,id)=>{
  e.stopPropagation();
  if(dragging)return;
  setHintDismissed(true);
  if(!selected){setSelected(id);return;}
  if(selected===id){setSelected(null);return;}
  const key=connKey(selected,id);
  setConnections(prev=>{
    const n=new Set(prev);
    n.has(key)?n.delete(key):n.add(key);
    return n;
  });
  setSelected(null);
};

const progress=level.required.filter(([a,b])=>connections.has(connKey(a,b))).le
const total=level.required.length;
const timePct=timeLeft/level.timeLimit;
const timerColor=timePct>0.5?T.green:timePct>0.25?T.yellow:T.red;

const CSS=`
  @import url('https://fonts.googleapis.com/css2?family=Cinzel:wght@700;900&dis
  @keyframes popIn{from{transform:scale(0.6);opacity:0}to{transform:scale(1);op
  @keyframes shimmer{0%,100%{opacity:0.7}50%{opacity:1}}
  @keyframes drift0{0%,100%{transform:translate(0,0)}50%{transform:translate(12
  @keyframes drift1{0%,100%{transform:translate(0,0)}50%{transform:translate(-1
  @keyframes drift2{0%,100%{transform:translate(0,0)}50%{transform:translate(8p
  @keyframes drift3{0%,100%{transform:translate(0,0)}50%{transform:translate(-1
  @keyframes pulseGlow{0%,100%{filter:drop-shadow(0 0 6px #c4a8f0)}50%{filter:d
```

```
  @keyframes cablePulse{0%{stroke-dashoffset:0}100%{stroke-dashoffset:-32}}
  @keyframes float{0%,100%{transform:translateY(0)}50%{transform:translateY(-6p
  @keyframes fadeSlide{from{opacity:0;transform:translateY(8px)}to{opacity:1;tr
  .dev-g{cursor:pointer;}
  .dev-g:hover rect,.dev-g:hover circle{filter:brightness(1.2);}
  .btn-tool{transition:all 0.2s;cursor:pointer;}
  .btn-tool:hover{transform:translateY(-1px);filter:brightness(1.15);}
  ::-webkit-scrollbar{width:4px;background:#0d0020;}
  ::-webkit-scrollbar-thumb{background:#4a1d8c;border-radius:4px;}
`;

// ── MENU ──────────────────────────────────────────────────────────────
if(screen==="menu"){
  return(
    <div style={{minHeight:"100vh",background:GRAD.bg,display:"flex",flexDirect
      <style>{CSS}</style>
      <Particles/>
      {/* decorative rings */}
      {[220,320,420].map((r,i)=>(
        <div key={i} style={{position:"absolute",left:"50%",top:"40%",transform
      ))}

      <div style={{position:"relative",textAlign:"center",animation:"popIn 0.6s
        {/* Logo */}
        <div style={{marginBottom:8,fontSize:64,animation:"float 3.5s ease-in-o
        <div style={{fontFamily:"'Cinzel',Georgia,serif",fontSize:"clamp(14px,2
        <h1 style={{fontFamily:"'Cinzel',Georgia,serif",fontSize:"clamp(28px,5v
          background:`linear-gradient(135deg, ${T.lavLight} 0%, ${T.blush} 50%,
          WebkitBackgroundClip:"text",WebkitTextFillColor:"transparent",letterS
          NETWORK<br/>CONNECTION
        </h1>
        <p style={{color:T.mutedDark,fontSize:12,letterSpacing:6,margin:"8px 0

        {/* Domain badge */}
        <div style={{display:"inline-flex",alignItems:"center",gap:8,
          background:"rgba(155,114,207,0.1)",border:`1px solid rgba(155,114,207
          borderRadius:30,padding:"6px 18px",marginBottom:32,
          boxShadow:`0 0 18px rgba(155,114,207,0.2)`}}>
          <span style={{width:7,height:7,borderRadius:"50%",background:T.green,
          <span style={{fontFamily:"'Courier New',monospace",fontSize:12,color:
            www.<span style={{color:T.blush,fontWeight:700}}>aounnet</span>.com
          </span>
        </div>

        {/* XP badge */}
        {totalXP>0&&<div style={{background:"rgba(155,114,207,0.15)",border:`1p
          ⭐ {totalXP} XP earned
```

```
        </div>}

        {/* Level cards */}
        <div style={{display:"flex",flexDirection:"column",gap:10,alignItems:"c
          {LEVELS.map((lvl,i)=>(
            <button key={lvl.id} className="btn-tool" onClick={()=>{setLevelIdx
              style={{width:310,padding:"14px 20px",background:i===0?GRAD.btn:"
                border:`1px solid ${i===0?"transparent":T.mutedDark}`,borderRad
                color:T.white,fontSize:13,textAlign:"left",display:"flex",align
              <span style={{fontSize:26}}>{lvl.badge}</span>
              <div>
                <div style={{fontWeight:700,fontFamily:"Georgia,serif",marginBo
                <div style={{fontSize:11,color:i===0?T.pinkLight:T.muted}}>{lvl
              </div>
              <div style={{marginLeft:"auto",fontSize:11,color:T.mutedDark}}>+{
            </button>
          ))}
        </div>

        <p style={{color:T.mutedDark,fontSize:11,marginTop:32,letterSpacing:2}}
        <p style={{color:"rgba(155,114,207,0.3)",fontSize:10,marginTop:10,lette
      </div>
    </div>
  );
}

// ── GAME ──────────────────────────────────────────────────────────────
return(
  <div style={{minHeight:"100vh",background:GRAD.bg,display:"flex",flexDirectio
    <style>{CSS}</style>
    <Particles/>

    {/* Top bar */}
    <div style={{display:"flex",alignItems:"center",justifyContent:"space-betwe
      background:GRAD.header,borderBottom:`1px solid rgba(155,114,207,0.2)`,
      backdropFilter:"blur(14px)",flexShrink:0,zIndex:10,position:"relative"}}>
      <div style={{display:"flex",alignItems:"center",gap:14}}>
        <button className="btn-tool" onClick={()=>setScreen("menu")} style={{ba
          ← Menu
        </button>
        <div>
          <div style={{color:T.white,fontFamily:"Georgia,serif",fontWeight:700,
            {level.badge} Level {level.id}: {level.title}
          </div>
          <div style={{display:"flex",alignItems:"center",gap:6,marginTop:2}}>
            <div style={{color:T.mutedDark,fontSize:10,letterSpacing:1}}>{level
            <span style={{color:T.mutedDark,fontSize:9}}>·</span>
```

```
        <span style={{fontFamily:"'Courier New',monospace",fontSize:9,color
      </div>
    </div>
  </div>

  <div style={{display:"flex",alignItems:"center",gap:10}}>
    {/* Tools */}
    {[["🔍","Ping",()=>setShowPing(true)],["📡","Trace",()=>setShowTrace(tr
      <button key={lbl} className="btn-tool" onClick={fn} style={{
        padding:"6px 13px",background:"rgba(155,114,207,0.12)",
        border:`1px solid rgba(155,114,207,0.3)`,borderRadius:9,
        color:T.lavLight,fontSize:12,display:"flex",alignItems:"center",gap
        {ico} <span style={{fontFamily:"'Courier New',monospace"}}>{lbl}</s
      </button>
    ))}

    {/* Timer */}
    <div style={{textAlign:"right",marginLeft:6}}>
      <div style={{fontSize:9,color:T.mutedDark,letterSpacing:2,marginBotto
      <div style={{fontSize:22,fontWeight:900,color:timerColor,fontFamily:"
        {String(Math.floor(timeLeft/60)).padStart(2,"0")}:{String(timeLeft%
      </div>
    </div>
  </div>
</div>

{/* Progress bar */}
<div style={{padding:"8px 18px",background:"rgba(26,9,51,0.5)",borderBottom
  <div style={{display:"flex",justifyContent:"space-between",fontSize:10,co
    <span>CONNECTIONS</span><span>{progress}/{total}</span>
  </div>
  <div style={{height:5,background:"rgba(155,114,207,0.12)",borderRadius:3,
    <div style={{height:"100%",borderRadius:3,background:GRAD.btn,transitio
  </div>
</div>

{/* Canvas */}
<div style={{flex:1,position:"relative",overflow:"hidden",zIndex:1}}>
  {/* dot grid */}
  <div style={{position:"absolute",inset:0,pointerEvents:"none",
    backgroundImage:`radial-gradient(circle, rgba(155,114,207,0.12) 1px, tr
    backgroundSize:"30px 30px"}}/>

  <svg ref={svgRef} style={{width:"100%",height:"100%",position:"absolute",
    onClick={()=>setSelected(null)}
    onMouseMove={onSvgMM} onMouseUp={onSvgMU} onMouseLeave={onSvgMU}>
    <defs>
```

```
  <filter id="devGlow"><feGaussianBlur stdDeviation="4" result="blur"/>
  <filter id="selGlow"><feGaussianBlur stdDeviation="6" result="blur"/>
  <linearGradient id="cableGrad" x1="0%" y1="0%" x2="100%" y2="0%">
    <stop offset="0%" stopColor={T.lavender}/>
    <stop offset="100%" stopColor={T.pink}/>
  </linearGradient>
</defs>

{/* Connections */}
{[...connections].map(key=>{
  const [aId,bId]=key.split("~~");
  const a=devices.find(d=>d.id===aId), b=devices.find(d=>d.id===bId);
  if(!a||!b)return null;
  const req=level.required.some(([x,y])=>connKey(x,y)===key);
  return(
    <g key={key}>
      {req&&<line x1={a.x} y1={a.y} x2={b.x} y2={b.y} stroke={T.lavende
      <line x1={a.x} y1={a.y} x2={b.x} y2={b.y}
        stroke={req?"url(#cableGrad)":T.cableBad}
        strokeWidth={req?2.5:1.5}
        strokeDasharray={req?"none":"5,4"}
        opacity={req?0.9:0.4}
      />
      {req&&<circle cx={(a.x+b.x)/2} cy={(a.y+b.y)/2} r={3.5} fill={T.b
    </g>
  );
})}

{/* Devices */}
{devices.map(dev=>{
  const cfg=DCONFIG[dev.type];
  const isSel=selected===dev.id;
  const s=cfg.size;
  const isRound=dev.type===DTYPE.ROUTER||dev.type===DTYPE.INTERNET;
  return(
    <g key={dev.id} className="dev-g"
      transform={`translate(${dev.x},${dev.y})`}
      onClick={e=>onDevClick(e,dev.id)}
      onMouseDown={e=>onDevMD(e,dev.id)}
      style={{animation:isSel?"pulseGlow 1.2s ease-in-out infinite":"no
      {/* Halo */}
      {isSel&&<ellipse cx={0} cy={0} rx={s+14} ry={s+14} fill="none" st
      {/* Shadow glow */}
      {isSel&&<ellipse cx={0} cy={0} rx={s+9} ry={s+9} fill={T.glowPink
      {/* Body */}
      {isRound
        ?<circle r={s} fill={cfg.color} stroke={isSel?T.blush:cfg.borde
```

```
              :<rect x={-s} y={-s} width={s*2} height={s*2} rx={9} fill={cfg.
            {/* Icon */}
            <text textAnchor="middle" dominantBaseline="middle" fontSize={s-6
            {/* Label */}
            <text textAnchor="middle" y={s+15} fontSize={9.5} fill={isSel?T.b
            {dev.ip&&<text textAnchor="middle" y={s+26} fontSize={8.5} fill={
          </g>
        );
      })}
    </svg>

    {/* Hint */}
    {!hintDismissed&&(
      <div style={{position:"absolute",bottom:20,left:"50%",transform:"transl
        background:"rgba(26,9,51,0.92)",border:`1px solid ${T.lavender}`,
        borderRadius:12,padding:"10px 22px",fontSize:12,color:T.lavLight,
        animation:"fadeSlide 0.4s ease",backdropFilter:"blur(10px)",whiteSpac
        💡 {level.hint}
      </div>
    )}

    {/* Selected label */}
    {selected&&(
      <div style={{position:"absolute",top:14,left:"50%",transform:"translate
        background:"rgba(155,114,207,0.18)",border:`1px solid ${T.lavender}`,
        borderRadius:20,padding:"7px 20px",fontSize:12,color:T.blush,
        backdropFilter:"blur(8px)",animation:"fadeSlide 0.3s ease",zIndex:20,
        {DCONFIG[devices.find(d=>d.id===selected)?.type]?.icon} 
        <b>{devices.find(d=>d.id===selected)?.label}</b> selected — click ano
      </div>
    )}

    {/* Timeout */}
    {timeLeft===0&&!showComplete&&(
      <div style={{position:"absolute",inset:0,background:"rgba(10,0,25,0.88)
        <div style={{fontSize:52,marginBottom:12}}>⌛</div>
        <h2 style={{color:T.red,fontFamily:"Georgia,serif",marginBottom:8}}>C
        <p style={{color:T.muted,fontSize:13,marginBottom:24}}>The network fa
        <div style={{display:"flex",gap:12}}>
          <button className="btn-tool" onClick={()=>initLevel(levelIdx)} styl
          <button className="btn-tool" onClick={()=>setScreen("menu")} style=
        </div>
      </div>
    )}
  </div>

  {/* Checklist sidebar */}
```

```
      <div style={{position:"absolute",right:0,top:90,bottom:0,width:170,
        background:"rgba(10,0,25,0.75)",borderLeft:`1px solid rgba(155,114,207,0.
        padding:"14px 12px",overflowY:"auto",backdropFilter:"blur(10px)",zIndex:1
        <div style={{fontSize:9,color:T.mutedDark,letterSpacing:3,marginBottom:12
        {level.required.map(([a,b])=>{
          const done=connections.has(connKey(a,b));
          const da=devices.find(d=>d.id===a), db=devices.find(d=>d.id===b);
          return(
            <div key={`${a}-${b}`} style={{display:"flex",alignItems:"flex-start"
              <span style={{fontSize:13,flexShrink:0}}>{done?"✅":"○"}</span>
              <span style={{fontFamily:"'Courier New',monospace",lineHeight:1.4}}
            </div>
          );
        })}
      </div>

      {/* Story overlay */}
      {showStory&&<StoryCard level={level} onStart={()=>setShowStory(false)}/>}

      {/* Modals */}
      {showPing&&<PingModal devices={devices} connections={connections} onClose={
      {showTrace&&<TraceModal devices={devices} onClose={()=>setShowTrace(false)}
      {showComplete&&(
        <LevelComplete
          level={level} timeLeft={timeLeft} timeLimit={level.timeLimit} xp={level
          isLast={levelIdx===LEVELS.length-1}
          onNext={()=>{const ni=levelIdx+1;initLevel(ni);setShowStory(true);}}
          onMenu={()=>{setShowComplete(false);setScreen("menu");}}
        />
      )}
    </div>
  );
}
```